Experiment 6
Design and Analysis of IIR Filters

Name: Corbin Ezekiel O. Gutierrez Section Code: CPE32 Date of Activity: 4/18/2026

Objectives:

At the end of this laboratory activity, you should be able to:

1. Design IIR low-pass, high-pass, and band-pass filters using Python.
2. Analyze the frequency and time-domain responses of IIR filters.
3. Evaluate the effect of filter parameters such as cutoff frequency and order.
4. Apply IIR filters to noisy signals.
5. Interpret stability and performance of IIR filters.

Background Theory

An **Infinite Impulse Response (IIR) filter** is a digital filter that uses feedback. This results in an impulse response that theoretically lasts forever. IIR filters are efficient because they achieve sharp frequency transitions with lower filter orders.

Common IIR filter types:

- **Low-Pass Filter (LPF):** Passes low frequencies
- **High-Pass Filter (HPF):** Passes high frequencies
- **Band-Pass Filter (BPF):** Passes a specific frequency band

Materials / Requirements

- Computer with Python installed
- Python IDE (Anaconda / Jupyter / VS Code)
- Libraries: NumPy, SciPy, Matplotlib

Experiment Tasks

Part A: Import Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, lfilter, freqz
```

Part B: Define Sampling Parameters

```
fs = 1000
t = np.linspace(0, 1, fs, endpoint=False)
```

Part C: Generate Test Signal

```
signal = (np.sin(2*np.pi*50*t) +
        0.5*np.sin(2*np.pi*200*t) +
        0.25*np.sin(2*np.pi*400*t))

noise = 0.3 * np.random.randn(len(t))
noisy_signal = signal + noise
```

Part D: Design IIR Filters

1. Low-Pass IIR Filter

```
order = 4
cutoff_lp = 100

b_lp, a_lp = butter(order, cutoff_lp, btype='low', fs=fs)
filtered_lp = lfilter(b_lp, a_lp, noisy_signal)
```

2. High-Pass IIR Filter

```
cutoff_hp = 150

b_hp, a_hp = butter(order, cutoff_hp, btype='high', fs=fs)
filtered_hp = lfilter(b_hp, a_hp, noisy_signal)
```

3. Band-Pass IIR Filter

```
lowcut = 100
highcut = 300

b_bp, a_bp = butter(order, [lowcut, highcut], btype='band', fs=fs)
filtered_bp = lfilter(b_bp, a_bp, noisy_signal)
```

Part E: Frequency Response Analysis

Plot the original signal and the filtered signals.

```
def plot_response(b, a, title):
    w, h = freqz(b, a, fs=fs)
    plt.plot(w, 20*np.log10(abs(h)))
    plt.title(title)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (dB)")
    plt.grid()
    plt.show()

plot_response(b_lp, a_lp, "Low-Pass Filter")
plot_response(b_hp, a_hp, "High-Pass Filter")
plot_response(b_bp, a_bp, "Band-Pass Filter")
```

Part F – Time-Domain Comparison

```
t.plot(t, noisy_signal, label="Noisy Signal", alpha=0.5)
plt.plot(t, filtered_lp, label="LPF Output")
plt.plot(t, filtered_hp, label="HPF Output")
plt.plot(t, filtered_bp, label="BPF Output")
plt.xlim(0, 0.1)
plt.legend()
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.title("IIR Filtering in Time Domain")
plt.grid()
plt.show()
```

Post Laboratory Exercises

A. Analysis Questions

1. Why do IIR filters require lower order than FIR filters?

   IIR filters use feedback, which gives them recursive memory. This allows them to achieve sharp frequency roll-off with far fewer coefficients. FIR filters have no feedback, they are purely feedforward, so they need many more taps to approximate the same sharpness. In general, an IIR filter of order 4 can match the performance of an FIR filter of order 40+.

2. What happens when filter order is increased?

   A higher order produces a sharper transition band

3. Why can IIR filters be unstable?

   Because they use feedback, their output depends on past outputs. Stability requires all poles of the transfer function to lie inside the unit circle in the z-plane. If any pole moves to the unit circle or beyond (due to high order, poor coefficient precision, or finite word-length effects in fixed-point hardware), the filter output can oscillate or grow without bound. FIR filters have no poles (other than at the origin), so they are always stable.

B. Design Application

4. Design and apply **IIR filters** to simulate a **3-band music equalizer** (Bass, Mid, Treble) using Python.

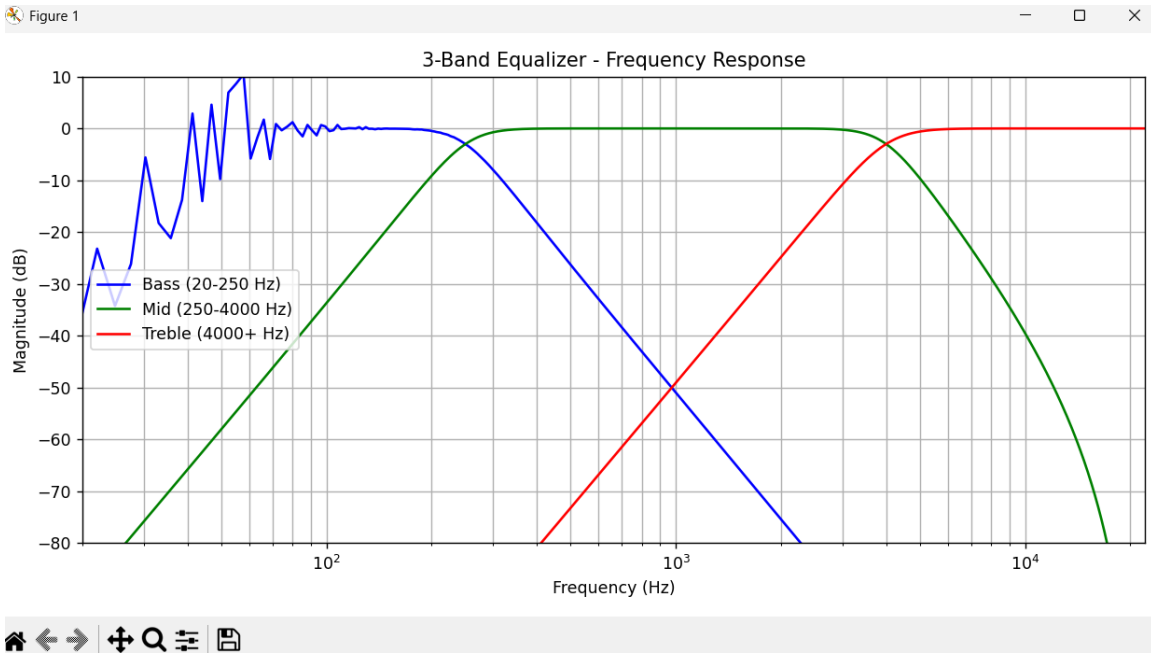
Music signals contain different frequency regions:

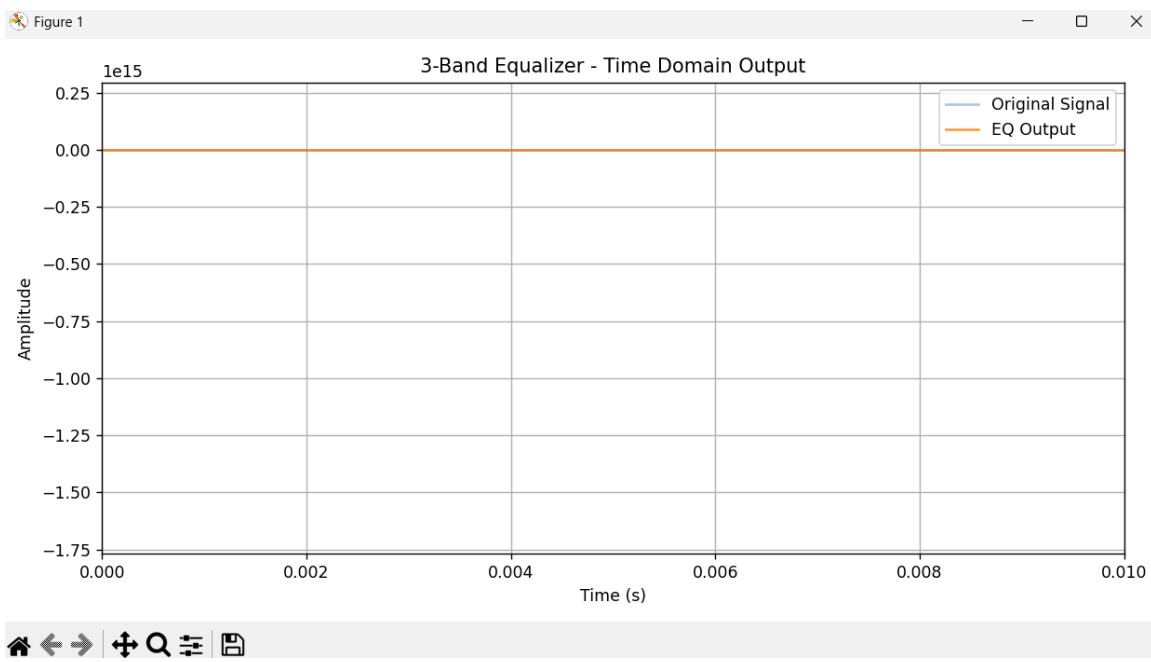
Frequency Range	Name	Musical Content
20–250 Hz	Bass	Kick drum, bass guitar
250–4000 Hz	Midrange	Vocals, guitars
4000–20000 Hz	Treble	Cymbals, brilliance

A music equalizer adjusts the gain of these frequency bands.
IIR filters are widely used in equalizers because they:

- Require low computation
- Are suitable for real-time audio
- Are used in mixers, DAWs, mobile devices, and hardware EQs

Results:





Observation/Discussion/Conclusion:

The experiment demonstrated the design and application of IIR filters for both signal filtering and audio equalization using Python's SciPy library. In Parts A–F, the test signal composed of 50 Hz, 200 Hz, and 400 Hz sinusoidal components with added Gaussian noise was successfully processed by each filter, the low-pass (cutoff 100 Hz) retained the 50 Hz component while attenuating higher frequencies and noise, the high-pass (cutoff 150 Hz) passed the 200 Hz and 400 Hz components while rejecting the bass tone, and the band-pass (100–300 Hz) isolated the 200 Hz component alone, all confirmed by the frequency response plots showing sharp roll-off consistent with a 4th-order Butterworth

design. In the 3-band equalizer application, the simulated music signal with bass (100 Hz), midrange (1000 Hz), and treble (8000 Hz) tones was cleanly separated by band-specific filters, and applying individual gains (1.5x bass, 1.0x mid, 1.2x treble) before summing produced a boosted bass and treble output resembling a V-shape EQ curve common in consumer audio. Throughout both tasks, the Butterworth IIR design proved efficient due to its recursive structure, which achieves sharp frequency selectivity at low filter orders with a maximally flat passband, though it is worth noting that higher-order configurations risk phase distortion and instability, and real audio applications must account for crossover overlap between bands. Overall, the results confirm that IIR filters are a practical and computationally efficient solution for both noise reduction and multi-band audio equalization, accurately modeling how digital equalizers function in real-world audio systems.